Om Dhumkekar

Test ID: 450014480000267 | 7972774586 | om.22310340@viit.ac.in

Test Date: September 26, 2025

Computer Science	Logical Ability	Node.js	Quantitative Ability (Advanced)
46 /100	42 /100	26 /100	52 /100

English Comprehension	Machine Learning	ReactJS	Automata Fix
44 /100	33 /100	25 /100	0 /100

Automata Pro	Personality
0 /100	Completed

Computer Science

46 / 100

OS and Computer Architecture	DBMS	Computer Networks
41 / 100	46 / 100	58 / 100

Logical Ability

42 / 100

Inductive Reasoning	Deductive Reasoning	Abductive Reasoning
46 / 100	35 / 100	45 / 100

© 2025 SHL and/or its affiliates. All rights reserved..

Node.js

26 / 100

Basics and Events	File Systems and Error Handling	DB, Server and Testing
52 / 100	26 / 100	0 / 100

Quantitative Ability (Advanced)

52 / 100

Basic Mathematics	Advanced Mathematics	Applied Mathematics
55 / 100	45 / 100	56 / 100

English Comprehension

44 / 100 | CEFR: **B1**

Grammar	Vocabulary	Comprehension
40 / 100	44 / 100	48 / 100

Machine Learning

33 / 100

Data Analysis and Statistics	Predictive Modelling	Advanced Machine Learning
25 / 100	40 / 100	33 / 100

ReactJS

25 / 100

React Basics	Components and Styling	Forms and Routing
25 / 100	0 / 100	67 / 100

Automata Fix

0 / 100

Logical Error	Code Reuse	Syntactical Error
0 / 100	0 / 100	0 / 100

Automata Pro

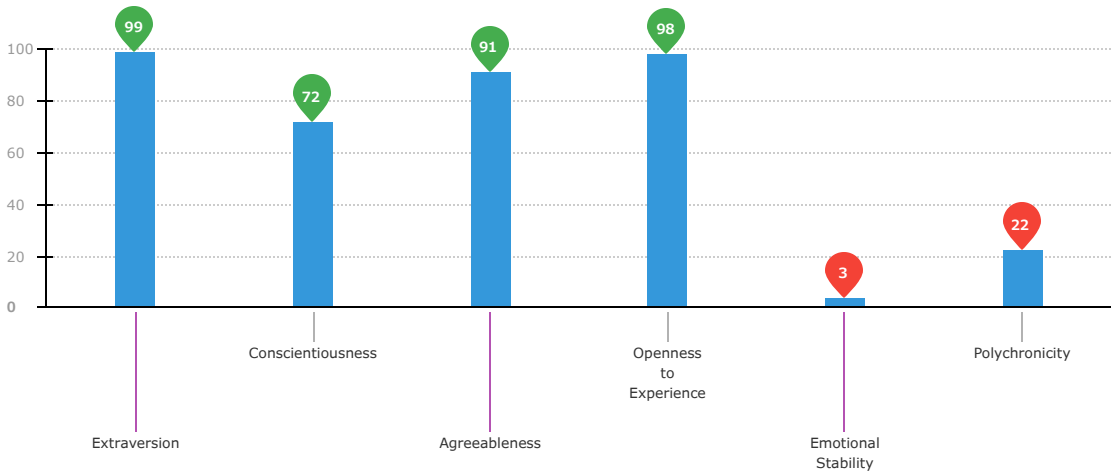
0 / 100

Programming Ability	Programming Practices	Functional Correctness
0 / 100	0 / 100	0 / 100

© 2025 SHL and/or its affiliates. All rights reserved..

Personality

Completed



Competencies

People Interaction

Self-Drive

Trainability

Repetitive Job Suitability

Work attributes

© 2025 SHL and/or its affiliates. All rights reserved..

1 | Introduction

About the Report

This report provides a detailed analysis of the candidate's performance on different assessments. The tests for this job role were decided based on job analysis, O*Net taxonomy mapping and/or criterion validity studies. The candidate's responses to these tests help construct a profile that reflects her/his likely performance level and achievement potential in the job role

This report has the following sections:

The **Summary** section provides an overall snapshot of the candidate's performance. It includes a graphical representation of the test scores and the subsection scores.

The **Insights** section provides detailed feedback on the candidate's performance in each of the tests. The descriptive feedback includes the competency definitions, the topics covered in the test, and a note on the level of the candidate's performance.

The **Response** section captures the response provided by the candidate. This section includes only those tests that require a subjective input from the candidate and are scored based on artificial intelligence and machine learning.

The **Learning Resources** section provides online and offline resources to improve the candidate's knowledge, abilities, and skills in the different areas on which s/he was evaluated.

Score Interpretation

All the test scores are on a scale of 0-100. All the tests except personality and behavioural evaluation provide absolute scores. The personality and behavioural tests provide a norm-referenced score and hence, are percentile scores. Throughout the report, the colour codes used are as follows:

- Scores between 67 and 100
- Scores between 33 and 67
- Scores between 0 and 33

© 2025 SHL and/or its affiliates. All rights reserved..

2 | Insights

English Comprehension

44 / 100 | CEFR: **B1**

This test aims to measure your vocabulary, grammar and reading comprehension skills.

You are able to state the meanings of English words used in day-to-day life. This is essential for you to read and understand short texts, write letters/emails, and understand internal communication at work.

Logical Ability

42 / 100

Inductive Reasoning

46 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

You are able to work out simple rules based on specific evidence or information. This skill is required in high end analytics jobs where one is required to infer patterns based on predefined rules from different sets of data.

Deductive Reasoning

35 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

You need to put in a great deal of effort to improve your predictive reasoning skills. Inductive reasoning will help you derive general rules from specific situations and carry out various tasks without needing instructions from others.

Abductive Reasoning

45 / 100

Quantitative Ability (Advanced)

52 / 100

This test aims to measure your ability to solve problems on basic arithmetic operations, probability, permutations and combinations, and other advanced concepts.

You are good at basic arithmetic. You are able to solve real-world problems that involve simple addition, subtraction, multiplication and division.

Personality

Competencies

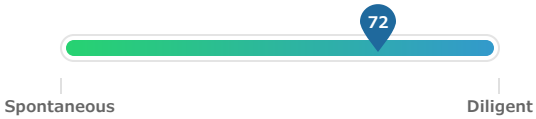
© 2025 SHL and/or its affiliates. All rights reserved..

Extraversion

Extraversion refers to a person's inclination to prefer social interaction over spending time alone. Individuals with high levels of extraversion are perceived to be outgoing, warm and socially confident.

- You are outgoing and seek out opportunities to meet new people.
- You tend to enjoy social gatherings and feels comfortable amongst strangers and friends equally.
- You display high energy levels and like to indulge in thrilling and exciting activities.
- You may tend to be assertive about your opinions and prefer action over contemplation.
- You take initiative and are more inclined to take charge than to wait for others to lead the way.
- Your personality is well suited for jobs demanding frequent interaction with people.

Conscientiousness



Conscientiousness is the tendency to be organized, hard working and responsible in one's approach to your work. Individuals with high levels of this personality trait are more likely to be ambitious and tend to be goal-oriented and focused.

- You value order and self discipline and tends to pursue ambitious endeavours.
- You believe in the importance of structure and is very well-organized.
- You carefully review facts before arriving at conclusions or making decisions based on them.
- You strictly adhere to rules and carefully consider the situation before making decisions.
- You tend to have a high level of self confidence and do not doubt your abilities.
- You generally set and work toward goals, try to exceed expectations and are likely to excel in most jobs, especially those which require careful or meticulous approach.

Agreeableness

Agreeableness refers to an individual's tendency to be cooperative with others and it defines your approach to interpersonal relationships. People with high levels of this personality trait tend to be more considerate of people around them and are more likely to work effectively in a team.

- You are considerate and sensitive to the needs of others.
- You tend to put the needs of others ahead of your own.
- You are likely to trust others easily without doubting their intentions.
- You are compassionate and may be strongly affected by the plight of both friends and strangers.
- You are humble and modest and prefer not to talk about personal accomplishments.
- Your personality is more suitable for jobs demanding cooperation among employees.

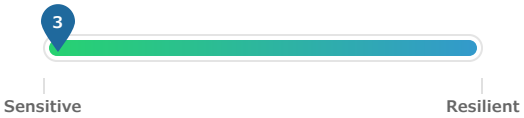
© 2025 SHL and/or its affiliates. All rights reserved..

Openness to Experience

Openness to experience refers to a person's inclination to explore beyond conventional boundaries in different aspects of life. Individuals with high levels of this personality trait tend to be more curious, creative and innovative in nature.

- You tend to be curious in nature and is generally open to trying new things outside your comfort zone.
- You may have a different approach to solving conventional problems and tend to experiment with those solutions.
- You are creative and tends to appreciate different forms of art.
- You are likely to be in touch with your emotions and is quite expressive.
- Your personality is more suited for jobs requiring creativity and an innovative approach to problem solving.

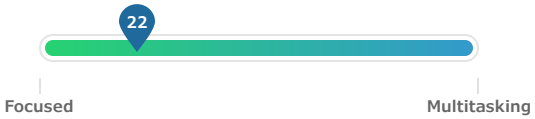
Emotional Stability



Emotional stability refers to the ability to withstand stress, handle adversity, and remain calm and composed when working through challenging situations. People with high levels of this personality trait tend to be more in control of their emotions and are likely to perform consistently despite difficult or unfavourable conditions.

- You are likely to be sensitive, emotional and may tend to worry about situations.
- You may react to everyday events with greater intensity and may become emotional.
- You may hesitate to face certain stressful situations and might feel anxious about your ability to handle them.
- You may find it hard to elicit self restraint and may tend to make impulsive decisions.
- Your personality is more suited for less stressful jobs.

Polychronicity



Polychronicity refers to a person's inclination to multitask. It is the extent to which the person prefers to engage in more than one task at a time and believes that such an approach is highly productive. While this trait describes the personality disposition of a person to multitask, it does not gauge their ability to do so successfully.

- You prefer to work on one task at a time, complete it and then move on to the next.
- You prefer orderliness and likes to concentrate on the task at hand without any distractions.
- You can find it difficult to be placed in a work environment where there is a need to multitask or where expected to engage in multiple projects simultaneously.

© 2025 SHL and/or its affiliates. All rights reserved..

Node.js

26 / 100

- You are aware of the basic syntax and structure of Node.js but needs to put in substantial effort to improve your conceptual knowledge and understanding of algorithms.
- You should start by trying to write small programs to improve your programming skills.

ReactJS

25 / 100

© 2025 SHL and/or its affiliates. All rights reserved..

3 | Response

Automata Pro

0 / 100 Code Replay

Question 1 (Language: Java 21)

An agent sends a message to headquarters containing the details of his project. He sends one soft copy to the agency's computer (P) and sends one hard copy by fax to Roger, the technical head of the agency (Q). During the transmission noise in the network causes some bits of the data message P to get distorted. However, we know that Roger always matches the binary values of both messages and checks whether he can convert the message P to message Q by flipping the minimum number of bits.

Write an algorithm to help Roger find the minimum number of bits that must be flipped to convert message P to message Q.

Scores

Programming Ability

0 / 100

Programming ability score cannot be generated. This is because source code has syntax/ runtime errors and is unparseable.

Functional Correctness

0 / 100

Syntactically incorrect code. The source code has syntax errors in it.

Final Code Submitted **Compilation Status: Fail**

```
// Sample code to read input and write output:

/*
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Scanner;

public class Solution
{
   public static void main(String args[] )
   {
      // Use either of these methods for input

      //BufferedReader
      BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
      String name1 = br.readLine();          // Read input from STDIN
      System.out.println("Hello " + name1);    // Write output to STDOUT

      //Scanner
      Scanner s = new Scanner(System.in);
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: O(1)

*N represents constant

Errors/Warnings

```
Compiling failed with exitcode 1, compiler output:
source_201.java:44: error: ';' expected
for(int result = 0;
result<-10^4<=num1,num2<=10^9; result++){
^
source_201.java:44: error: not a statement
for(int result = 0;
result<-10^4<=num1,num2<=10^9; result++){
^
source_201.java:44: error: ')' expected
```

© 2025 SHL and/or its affiliates. All rights reserved..

```

    String name2 = s.nextLine();            // Read input from STDIN
    System.out.println("Hello " + name2);    // Write output to STDOUT
  }
}
*/

// Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Scanner;
class Solution
{
  public static void main(String args[] )
  {
    // Write your code here
    Scanner in = new Scanner(System.in);


    int num1 = in.nextInt();
    int num2 = in.nextInt();
    int num2 = minNoOfBits.num1;

    for(int result = 0; result<-10^4<=num1,num2<=10^9; result++){

    }

    int result = num1.minNoOfBits(num2);

    System.out.println("Intger representing the minimum number of bits that must be flipped to convert message P to Q" + result);
  }

  public static int minNoOfBits(int num1, int num2, int result){
    int result = 0;

    return result;
  }
}
```

```
for(int result = 0;
result<-10^4<=num1,num2<=10^9; result++){
^
source_201.java:44: error: ';' expected
for(int result = 0;
result<-10^4<=num1,num2<=10^9; result++){
^
4 errors
```

© 2025 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

3	1	2	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:25:59**
Average time taken between two compile attempts:	**00:08:40**
Average test case pass percentage per compile:	**5.13%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Java 21)

A convention venue is hosting a meeting of large firms, which N number of CEOs will be attending. Each CEO is assigned an invitation ID from 0 to N-1. Each CEO has some CEOs whom they like best. They will attend the meeting only if they can be seated next to the person they like. You are asked to plan this seating arrangement.

Write an algorithm to find the maximum number of CEOs who will attend the meeting.

© 2025 SHL and/or its affiliates. All rights reserved..

Scores

Programming Ability

0 / 100

NA

Programming Practices

0 / 100

Programming practices score cannot be generated. This is because source code has syntax/runtime errors and is unparseable or the source code does not meet the minimum code-length specifications.

Functional Correctness

0 / 100

NA

Final Code Submitted

Compilation Status: Pass

```
// Sample code to read input and write output:

/*
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Scanner;

public class Solution
{
   public static void main(String args[] )
   {
      // Use either of these methods for input

      //BufferedReader
      BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
      String name1 = br.readLine();          // Read input from STDIN
      System.out.println("Hello " + name1);    // Write output to STDOUT

      //Scanner
      Scanner s = new Scanner(System.in);
      String name2 = s.nextLine();           // Read input from STDIN
      System.out.println("Hello " + name2);    // Write output to STDOUT
   }
}
*/

// Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: O(N)

*N represents number of CEO

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

© 2025 SHL and/or its affiliates. All rights reserved..

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Scanner;
class Solution
{
    public static void main(String args[] )
    {
        // Write your code here
    }
}
```

Test Case Execution

Passed TC: **0%**

Total score

0/13

0%	0%	0%
Basic(**0**/8)	Advance(**0**/3)	Edge(**0**/2)

Compilation Statistics

0	0	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:41**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**0%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

© 2025 SHL and/or its affiliates. All rights reserved..

Automata Fix

0 / 100 Code Replay

Question 1 (Language: Java)

The function/method ***calculateMatrixSum*** returns an integer representing the sum of odd elements of the given matrix whose i^{th} and j^{th} index are the same.

The function/method ***calculateMatrixSum*** accepts three arguments - *rows*, an integer representing the number of rows of the given matrix, *columns*, an integer representing the number of columns of the given matrix and *matrix*, representing a two-dimensional array of integers.

The function/method ***calculateMatrixSum*** compiles successfully but fails to return the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Pass

```
// You can print the values to stdout for debugging
class Solution
{
   int calculateMatrixSum(int rows, int columns, int matrix[][])
   {
      int i, j, sum=0;
      if((rows>0) && (columns>0))
      {
         for(i=0;i<rows;i++)
         {
            sum =0;
            for(j=0;j<columns;j++)
            {
               if(i==j)
               {
                  if(matrix[i][j]/2!=0)
                     sum += matrix[i][i];
               }
            }
         }
         return sum;
      }
      else
         return sum;
   }
}
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

© 2025 SHL and/or its affiliates. All rights reserved..

Test Case Execution

Passed TC: **12.5%**

Total score

1/8

0%	0%	100%
Basic(**0**/4)	Advance(**0**/3)	Edge(**1**/1)

Compilation Statistics

1	1	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:02:49**
Average time taken between two compile attempts:	**00:02:49**
Average test case pass percentage per compile:	**0%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Java)

The function/method ***sortArray*** modify the input list by sorting its elements in descending order.
The function/method ***sortArray*** accepts two arguments - *len,* representing the length of the list and *arr,* a list of integers representing the input list, respectively.

The function/method ***sortArray*** compiles successfully but fails to get the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

© 2025 SHL and/or its affiliates. All rights reserved..

Scores

Final Code Submitted

Compilation Status: Pass

```
// You can print the values to stdout for debugging
class Solution
{
   public void sortArray(int len, int[] arr)
   {
      int i , max , location , j , temp;
      for( i = 0 ; i < len ; i ++ )
      {
         max = arr[i];
         location = i;
         for( j = i ; j < len ; j ++ )
         {
            if( max > arr[j] )
            {
               max = arr[j];
               location = j;
            }
         }
         temp = arr[i];
         arr[i] = arr[location];
         arr[location] = temp;
      }
   }
}
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **25%**

Total score

2/8

33%	0%	100%
Basic(**1**/3)	Advance(**0**/4)	Edge(**1**/1)

Compilation Statistics

0	0	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:12**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**25%**

© 2025 SHL and/or its affiliates. All rights reserved..

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 3 (Language: Java)

The function/method ***allExponent*** returns a real number representing the result of exponentiation of base raised to power exponent for all input values. The function/method ***allExponent*** accepts two arguments - *baseValue*, an integer representing the base and *exponentValue*, an integer representing the exponent.

The incomplete code in the function/method ***allExponent*** works only for positive values of the exponent. You must complete the code and make it work for negative values of exponent as well.

Another function/method ***positiveExponent*** uses an efficient way for exponentiation but accepts only positive *exponent* values. You are supposed to use this function/method to complete the code in ***allExponent*** function/method.

Helper Description

The following class is used to represent a Exponent and is already implemented in the default code (Do not write this definition again in your code):

```
public class Exponent

{

   public int base;

   public int exponent;

   int positiveExponent()

   {

      /*It calculate the Exponent for positive value of exponentValue
```

© 2025 SHL and/or its affiliates. All rights reserved..

```
        This can be called as -

        Exponent exp = new Exponent(baseValue, exponentValue);

        float res = exp.positiveExponent();*/

    }

}
```

Scores

Final Code Submitted

Compilation Status: Pass

```
1  // You can print the values to stdout for debugging
2  class Solution
3  {
4     float allExponent(int baseValue, int exponentValue)
5     {
6        float res = 1;
7        if(exponentValue >=0)
8        {
9           Exponent exp = new Exponent(baseValue, exponentValue);
10          res = (float)exp.positiveExponent();
11       }
12       else
13       {
14          // write your code here for negative exponentInput
15
16       }
17       return res;
18    }
19 }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **66.67%**

Total score

4/6

100%	33%	100%
Basic(**2**/2)	Advance(**1**/3)	Edge(**1**/1)

© 2025 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

0	0	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:54**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**66.7%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 4 (Language: Java)

The function/method ***replaceValues*** is modifying the input list in such a way - if the length of input list is odd, then all the elements of the input list are supposed to be replaced by 1s and in case it is even, the elements should be replaced by 0s.

For example: given the input list [0 1 2], the function will modify the input list like [1 1 1]

The function/method ***replaceValues*** accepts two arguments - *size*, an integer representing the size of the given input list and *inputList*, a list of integers representing the input list.

The function/method ***replaceValues*** compiles successfully but fails to get the desired result for some test cases due to incorrect implementation of the function. Your task is to fix the code so that it passes all the test cases.

© 2025 SHL and/or its affiliates. All rights reserved..

Scores

Final Code Submitted

Compilation Status: Pass

```
// You can print the values to stdout for debugging
class Solution
{
  void  replaceValues(int size, int[] inputList)
  {
    int i , j;
    if( size % 2 == 0 )
    {
      i=0;
      while(i<size)
      {
        inputList[i] = 0;
        i+=2;
      }
    }
    else
    {
      j=0;
      while(j<size)
      {
        inputList[j] = 1;
        j+=2;
      }
    }
  }
}

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **25%**

Total score

2/8

0%	0%	100%
Basic(**0**/6)	Advance(**0**/0)	Edge(**2**/2)

© 2025 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

0	0	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:00**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**25%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 5 (Language: Java)

You are given predefined structure ***Time*** containing *hour*, *minute*, and *second* as members. A collection of functions/methods for performing some common operations on times is also available. You must make use of these functions/methods to calculate and return the difference.

The function/method ***difference_in_times*** accepts two arguments - *time1*, and *time2*, representing two times and is supposed to return an integer representing the difference in the number of seconds.

You must complete the code so that it passes all the test cases.

.

Helper Description

© 2025 SHL and/or its affiliates. All rights reserved..

The following class is used to represent a Time and is already implemented in the default code (Do not write this definition again in your code):

```
public class Time

{

   public int hour;

   public int minute;

   public int second;

   public Time(int h, int m, int s)

   {

      hour = h;

      minute = m;

      second = s;

   }

   public int compareTo(Object anotherTime)

   {

      /*Return 1, if time1 is greater than time2.

      Return -1 if time1 is less than time2

      or, Return 0, if time1 is equal to time2

      This can be called as -

      *  If time1 and time2 are two Time then -

      *  time1.compareTo(time2) */

   }

   public void addSecond()

   {

      /* Add one second in the time;

      This can be called as -

      *  If time1 is Time then -

      *  time1.addSecond() */

   }
```

© 2025 SHL and/or its affiliates. All rights reserved..

Scores

Final Code Submitted

Compilation Status: Fail

```
// You can print the values to stdout for debugging
class Solution
{
   int difference_in_times(Time time1, Time time2)
   {
      // write your code here
   }
}

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

```
Solution.java:7: error: missing return statement
}
^
Time.java:5: warning: [rawtypes] found raw type: Comparable
public class Time implements Comparable
^
missing type arguments for generic class Comparable
where T is a type-variable:
T extends Object declared in interface Comparable
1 error
1 warning
```

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Compilation Statistics

Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors
0	0	0	0	0	0

Response time:	**00:01:15**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**0%**

© 2025 SHL and/or its affiliates. All rights reserved..

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 6 (Language: Java)

The function/method ***productMatrix*** accepts three arguments - *rows,* an integer representing the rows of the matrix; *columns,* an integer representing the columns of the matrix and *matrix*, a two-dimensional array of integers, respectively.

The function/method ***productMatrix*** return an integer representing the product of the odd elements whose i^{th} and j^{th} index are the same. Otherwise, it returns 0.

The function/method ***productMatrix*** compiles successfully but fails to return the desired result for some test cases. Your task is to debug the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Pass**

```
public class Solution
{
  public int productMatrix(int rows, int columns, int matrix[][])
   {
      int result=0;
      for(int i=0;i<rows;i++)
         for(int j=0;j<columns;j++)
            if((i==j) || (matrix[i][j]%2!=0))
               result *=matrix[i][j];
      if(result<=1)
         return 0;
      else
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

© 2025 SHL and/or its affiliates. All rights reserved..

```
        return result;
    }
}
```

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **25%**

Total score 2/8

0%	100%	0%
Basic(**0**/5)	Advance(**2**/2)	Edge(**0**/1)

Compilation Statistics

0	0	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:00**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**25%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 7 (Language: Java)

The function/method *maxReplace* prints space separated integers representing the input list after replacing all

© 2025 SHL and/or its affiliates. All rights reserved..

elements of the input list with the maximum element of the input list.

The function/method ***maxReplace*** accept two arguments - *size*, an integer representing the size of the input list and *inputList,* a list of integers representing the input list, respectively.

The function/method ***maxReplace*** compiles unsuccessfully due to compilation error. Your task is to debug the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Fail**

```
// You can print the values to stdout for debugging
class Solution
{
   void maxReplace(int size, int &inputList)
   {
      if(size>0)
      {
         int max =inputList[0];
         for(int i=0;i<size;i++)
         {
            if(max<inputList[i])
            {
               max = inputList[i];
            }
         }
      }
      for(int i=0;i<size,i++)
      {
         inputList[i]=max
         System.out.print(inputList[i]+" ");
      }
   }
}
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

```
Solution.java:4: error: expected
void maxReplace(int size, int &inputList)
^
Solution.java:4: error: ';' expected
void maxReplace(int size, int &inputList)
^
Solution.java:4: error: illegal start of type
void maxReplace(int size, int &inputList)
^
Solution.java:4: error: expected
void maxReplace(int size, int &inputList)
^
Solution.java:5: error: ';' expected
{
^
Solution.java:6: error: illegal start of type
if(size>0)
^
Solution.java:6: error: ';' expected
if(size>0)
^
Solution.java:17: error: illegal start of type
for(int i=0;i ^
Solution.java:17: error: ')' expected
for(int i=0;i ^
Solution.java:17: error: illegal start of type
for(int i=0;i ^
```

© 2025 SHL and/or its affiliates. All rights reserved..

```
Solution.java:17: error: expected
for(int i=0;i ^
Solution.java:17: error: ';' expected
for(int i=0;i ^
Solution.java:17: error: > expected
for(int i=0;i ^
Solution.java:17: error: expected
for(int i=0;i ^
Solution.java:18: error: '(' expected
{
^
Solution.java:19: error: ']' expected
inputList[i]=max
^
Solution.java:19: error: ';' expected
inputList[i]=max
^
Solution.java:20: error: ';' expected
System.out.print(inputList[i]+" ");
^
Solution.java:20: error: expected
System.out.print(inputList[i]+" ");
^
Solution.java:20: error: ']' expected
System.out.print(inputList[i]+" ");
^
Solution.java:20: error: ')' expected
System.out.print(inputList[i]+" ");
^
Solution.java:20: error: illegal start of type
System.out.print(inputList[i]+" ");
^
Solution.java:20: error: expected
System.out.print(inputList[i]+" ");
^
Solution.java:20: error: ';' expected
System.out.print(inputList[i]+" ");
^
Solution.java:22: error: class, interface, or enum
expected
}
^
25 errors
```

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

© 2025 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

0	0	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:01**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**0%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

© 2025 SHL and/or its affiliates. All rights reserved..

4 | Learning Resources

English Comprehension

- Read about how to communicate effectively in english
- Practice english grammar
- Learn about the basics of reading

Logical Ability

- Learn about using deduction to arrive at valid conclusions
- Learn about validity of arguments
- Test your inductive logic!

Quantitative Ability (Advanced)

- Learn about percentages
- Learn about simple and compount interests
- Watch a video on time, speed and distance

Icon Index

Free Tutorial	Paid Tutorial	Youtube Video	Web Source
Wikipedia	Text Tutorial	Video Tutorial	Google Playstore

© 2025 SHL and/or its affiliates. All rights reserved..